

Branching Policy: 'Git/Git Flow' Based Versioning

Patrick TOCA - D&T CBRE Ltd

October 14th, 2025 ⁽¹⁾

This document describes the branching policy of the "D&T" team, and presents also how to use it on a day to day basis.

The objectives of this document are:

- a. to describe the fundamentals principles of the Git/Git Flow versioning process,
- b. to present the added values presented by this Git/Git Flow based process,
- c. to provide sufficient guidelines to help everyone in the team to get started and become proficient in short time.

This handout is aimed at the members of the "D&T" team(s) and has been distributed before the training session on Git/Git Flow held on January 2019.

Outline of the hand-out

1. Advantages of the Git/Git Flow based process
2. General concepts of versioning with Git/Git Flow
3. Branching terminology and what it implies
4. Git Flow commands in practice
5. Conclusion
6. Useful links

1 Advantages of the Git/Git Flow Based Process

Git/Git Flow facilitates the realization of a simple working process within the development team.

It enforces certain practices such as naming conventions for your branches and assists with the flow from development, to release, to master.

Git/Git Flow doesn't offer anything that basic git commands can't do.

With the 'start' and 'finish' commands Git Flow only abstracts away the merging and deleting Git native commands, a simplification.

Benefits are:

- enforces a process (flow) by having known branch types: master, develop, feature, release, hotfix
- standardises branch naming conventions
- abstracts certain git commands for merging and deleting (See link here: [Git and Git Flow commands comparison: a simplification](#))
- standardises our branching policy on validated practices.

¹ first created on 20th December, 2018

2 General Concepts of Versioning with Git/Git Flow

The document: [Git the essential concepts](#) presents in detail what we introduce here.

Git stores and thinks about information in a very different way than other VCS (Subversion, etc.), and understanding these differences will help you avoid becoming confused while using it.

2.a Snapshots, Not Differences

Git thinks of its data more like a series of snapshots of a miniature filesystem. With Git, every time you commit, it saves the state of your project.

To be efficient, if files have not changed, Git doesn't store the file again, just a link to the previous identical file it has already stored. Git thinks about its data more like a stream of snapshots.

It makes Git reconsider almost every aspect of version control that most other systems copied from the previous generation.

Git has been one of the very first fully distributed applications ever delivered in production on a large scale.

NEED TO KNOW: Version Control System provides facilities to model files changing over time and distribute different versions efficiently.

The popular version control system Git provides a powerful Merkle DAG object model that captures changes to a filesystem tree in a distributed-friendly way.

1. **Immutable objects** represent Files (blob), Directories (tree), and Changes (commit).
2. Objects are **content-addressed**, by the cryptographic hash of their contents.
3. **Links to other objects are embedded**, forming a Merkle DAG.
This provides many useful integrity and workflow properties.
4. Most versioning metadata (branches, tags, etc.) are simply pointer references, and thus inexpensive to create and update.
5. Version changes only update references or add objects.
6. Distributing version changes to other users is simply transferring objects and updating remote references.(see nota bene ²)

² Git is one of the first fully distributed, highly concurrent and scalable applications by design.

2.b Nearly All Operations Are Local

Most operations in Git need only local files and resources to operate; generally no information is needed from another computer on your network. Because you have the entire history of the project right there on your local disk, most operations seem almost instantaneous.

This also means that there is very little you can't do if you're offline or off VPN.

2.c Git Has Integrity

Everything in Git is checksummed before it is stored and is then referred to by that checksum. This means it's impossible to change the contents of any file or directory without Git knowing about it.

This functionality is built into Git at the lowest levels and is integral to its philosophy.

The mechanism that Git uses for this checksumming is called a SHA-1 hash.

A SHA-1 hash looks something like this:

```
24b9da6552252987aa493b52f8696cd6d3boo373
```

In fact, Git stores everything in its database not by file name but by the hash value of its contents.

2.d Git Generally Only Adds Data

When you do actions in Git, nearly all of them only add data to the Git database.

It is hard to get the system to do anything that is not undoable or to make it erase data in any way.

See here: [To go further on Undoing things with Git.](#)

2.e Git Manages 3 States

Pay attention now! Here is the main thing to remember about Git if you want the rest of your learning process to go smoothly.

Git has three main states that your files can reside in: committed, modified, and staged:

- **Committed** means that the data is safely stored in your local database.
 - **Modified** means that you have changed the file but have not committed it to your database yet.
 - **Staged** means that you have marked a modified file in its current version to go into your next commit snapshot.
-

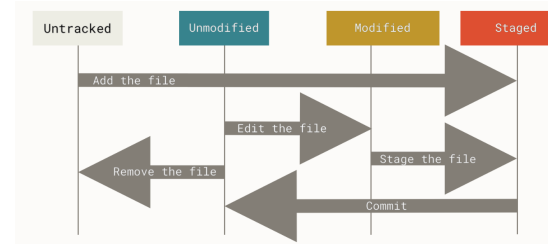


Figure 8. The lifecycle of the status of your files

Figure 1: The 3 Git stages - Working tree, staging and Git directory

4 *Gitflow commands in practice*

Gitflow simplifies the Git versioning branching process to a few set of operations (see nota bene ¹) : Initialise, Connect, Features, Releases

¹ Nota Bene: Using the 'showcom-
mands' switch and Hotfix. you can
see the actual git commands being
executed by the Gitflow plugin : [git
flow feature start FEATURE-NAME
-showcommands](#)

4.a *Initialise*

Gitflow	Git
git flow init	git init git commit -allow-empty -m "Initial commit" git checkout -b develop master

Table 1: Git Flow reduces significantly the number of commands.

git flow init also edits your .git/config and prompts for values:

[gitflow "branch"]

- master = master
- develop = develop

[gitflow "prefix"]

- feature = feature
- release = release
- hotfix = hotfix
- support = support
- versiontag =

4.b *Connect to Remote repository*

Gitflow	Git
git flow init	git init
Already done during the init	git remote add origin git@github.com:MYACCOUNT/MYREPO

4.c Features branching

Creating a feature branch

Gitflow	Git
git flow feature start MYFEATURE	git checkout -b feature/MYFEATURE develop

Sharing a feature branch

Gitflow	Git
git flow feature publish MYFEATURE	git checkout feature/MYFEATURE git push origin feature/MYFEATURE

Get latest for a feature branch

Gitflow	Git
git flow feature pull origin MYFEATURE	git init git checkout feature/MYFEATURE git pull --rebase origin feature/MYFEATURE

Finalize a feature branch

Gitflow	Git
git flow feature finish MYFEATUR	git checkout develop git merge --no-ff feature/MYFEATURE git branch -d feature/MYFEATURE

Push the merged feature branch

Gitflow	Git
Already done during the finish	git push origin develop git push origin :feature/MYFEATURE (if pushed)

*4.d Releases branching***Create a release branch**

Gitflow

Git

 git flow release start **v1.2.0** git checkout -b release/v1.2.0 develop
Share a release branch

Gitflow

Git

 git flow release publish **v1.2.0** git checkout -b **release/v1.2.0** develop
Get latest for a release branch

Gitflow

Git

 Already done during the finish git checkout -b **release/v1.2.0** develop
 git pull --rebase origin **release/v1.2.0**

Finalize a release branch

Gitflow	Git
<code>git flow release finish v1.2.0</code>	<code>git checkout master</code> <code>git pull --rebase origin release/v1.2.0</code> <code>git merge --no-ff release/v1.2.0</code> <code>git tag -a v1.2.0 \$RELEASE_BRANCH</code> (to include the branch name) <code>git checkout develop</code> <code>git merge --no-ff release/v1.2.0</code> <code>git branch -d release/v1.2.0</code>

Push the merged release branch

Gitflow	Git
<code>Already done during the finish</code>	<code>git push origin master</code> <code>git push origin develop</code> <code>git push origin --tags</code> <code>git push origin:release/v1.2.0</code> (if pushed)

*4.e Hotfixes branching***Create a hotfix branch**

Gitflow	Git
<code>git flow hotfix start v1.2.1 [commit]</code>	<code>git checkout -b hotfix/v1.2.1 [commit]</code>

Finalize a hotfix branch

Gitflow	Git
<code>git flow hotfix finish v1.2.1</code>	<code>git checkout master</code> <code>git merge --no-ff hotfix/v1.2.1</code> <code>git tag -a v1.2.1</code> <code>git checkout develop</code> <code>git merge --no-ff hotfix/v1.2.1</code> <code>git branch -d hotfix/v1.2.1</code>

Push the merged hotfix branch

Gitflow	Git
Already done during the finish	<code>git push origin master</code> <code>git push origin develop</code> <code>git push origin --tags</code> <code>git push origin:hotfix/v1.2.1 (if pushed)</code>

5 Conclusion

Git-Flow is an extremely powerful workflow for doing development and seems to cater for just about every scenario you can come across in business. It is simple to follow once you start using it and makes for a good standard across small and big companies. If you use it in conjunction with Git you'll quickly realize the strength of it (combined with Test Driven Development practices, Continuous Integration as well as Regression Testing scenarios) while not even taking into consideration the usual effort that goes into doing a Release or even a Hotfix.

Parallel Development

One of the great things about GitFlow is that it makes parallel development very easy, by isolating new development from finished work. New development (such as features and non-emergency bug fixes) is done in feature branches, and is only merged back into main body of code when the developer(s) is happy that the code is ready for release. Although interruptions are a BadThing(tm), if you are asked to switch from one task to another, all you need to do is commit your changes and then create a new feature branch for your new task. When that task is done, just checkout your original feature branch and you can continue where you left off.

Collaboration Feature

Branches also make it easier for two or more developers to collaborate on the same feature, because each feature branch is a sandbox where the only changes are the changes necessary to get the new feature working. That makes it very easy to see and follow what each collaborator is doing.

Release Staging Area

As new development is completed, it gets merged back into the develop branch, which is a staging area for all completed features that haven't yet been released. So when the next release is branched off of develop, it will automatically contain all of the new stuff that has been finished.

Support For Emergency Fixes

GitFlow supports hotfix branches - branches made from a tagged release. You can use these to make an emergency change, safe in the knowledge that the hotfix will only contain your emergency fix. There's no risk that you'll accidentally merge in new development at the same time.

5.a *Useful links*

1. [a graphic representation of the Gitflow process](#)
2. [Vincent Driessen's original blog post about Gitflow - 2010](#)
3. [Gitflow for Github: further consolidation](#)
4. [Gitflow animation](#)
5. [Finally, to install Gitflow on your laptop](#)